

Wprowadzenie do projektowanie serwisów internetowych

Wykłady 1

Bartosz Lauks

4 marca 2026



- 1 Historia
- 2 Wprowadzenie
- 3 Front-end
- 4 Back-end
- 5 Technologie wspierające

- 1 Historia
- 2 Wprowadzenie
- 3 Front-end
- 4 Back-end
- 5 Technologie wspierające

World Wide Web (WWW)

Historia

- Pomysł stworzenia **World Wide Web** zrodził się w **1989** roku, kiedy **Tim Berners-Lee**, pracując w **CERN** (Europejskiej Organizacji Badań Jądrowych) w Szwajcarii, dostrzegł problem w organizacji wymiany informacji między naukowcami.
- W **CERN** znajdowały się dane naukowe, które były przechowywane na różnych komputerach, ale nie było efektywnego sposobu ich udostępniania i przeszukiwania.
- **Berners-Lee** zaproponował stworzenie systemu, który łączyłby dane z różnych komputerów w sieci, wykorzystując **hiperteksty**. To właśnie był początek **WWW**.

World Wide Web (WWW)

Pierwsza wersja WWW i jej działanie

- **Berners-Lee** stworzył pierwszą stronę internetową w **1991** roku, a jego pierwsza wersja systemu **WWW** składała się z kilku elementów:
 - **Pierwsza przeglądarka** (Nexus) - "**WorldWideWeb**" (przeglądarka i edytor HTML w jednym), która była również pierwszą stroną internetową i pozwalała na tworzenie oraz przeglądanie stron.
 - **HTTP** – protokół przesyłania dokumentów przez internet.
 - **HTML** – język znaczników służący do tworzenia dokumentów tekstowych, które mogły zawierać **hiperłącza**.
- Pierwsza strona na komputerze **NeXT**, który służył jako **serwer WWW**.

World Wide Web (WWW)

Pierwszy serwer WWW na komputerze NeXT

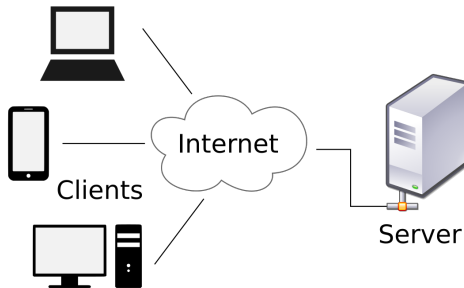
- **Serwer WWW** działał na tym komputerze i był wewnętrzny – dostępny tylko w sieci **CERN**.
- Ponieważ w początkowej fazie **WWW** było to zamknięte środowisko, a **Internet** w dzisiejszym sensie nie istniał jeszcze w pełni.
- Przełomem w projekcie było połączenie hipertekstu z Internetem
- Kolejnym krokiem na potrzeby projektu było opracowanie ogólnodostępnych, unikatowych identyfikatorów zasobów sieci: „The Universal Document Identifier” (**UDI**) znany później jako Uniform Resource Locator (**URL**) i Uniform Resource Identifier (**URI**)

- 1 Historia
- 2 Wprowadzenie
- 3 Front-end
- 4 Back-end
- 5 Technologie wspierające

Architektura klient serwer

Czym jest ?

- **Architektura klient-serwer** to model komunikacji w systemach komputerowych, w którym jedna strona (**klient**) wysyła żądania do drugiej strony (**serwera**), a serwer odpowiada na te żądania.
- Klient inicjuje interakcję, natomiast serwer dostarcza zasoby lub usługi.
- Cechy architektury klient-serwer:
 - **Podział ról** – klient odpowiada za interakcję z użytkownikiem, a serwer za przetwarzanie żądań i przechowywanie danych.
 - **Modularność** – klient i serwer mogą działać na różnych maszynach i systemach operacyjnych.
 - **Skalowalność** – system można rozszerzać poprzez dodawanie nowych klientów lub serwerów.
 - **Centralizacja danych** – dane przechowywane są na serwerze, co ułatwia zarządzanie i zabezpieczanie informacji.



Rysunek 1: Architektura client-server

Serwer, Serwer WWW oraz Serwer HTTP

Czym jest Serwer ?

- **Serwer** to komputer lub program, który świadczy usługi lub przechowuje zasoby dostępne dla innych urządzeń.

Czym jest Serwer WWW oraz Serwer HTTP ?

- **Serwer WWW** (World Wide Web) – to ogólna nazwa dla serwera, który przechowuje zasoby internetowe, takie jak strony HTML, obrazy, pliki CSS, JavaScript i inne elementy, które są wyświetlane w przeglądarkach internetowych.
- **Serwer HTTP** (Hypertext Transfer Protocol) – to serwer HTTP to program, który nasłuchuje na odpowiednich portach (zwykle port 80) i obsługuje zapytania HTTP, takie jak pobieranie stron internetowych lub wysyłanie danych formularzy.

Hosting

Czym jest ?

- **Hostowanie** (hosting) to usługa, która umożliwia przechowywanie stron internetowych, aplikacji, plików i danych na serwerach dostępnych przez Internet.
- Dzięki hostowaniu, Twoja strona może być dostępna online dla innych użytkowników, którzy mogą ją odwiedzać za pomocą przeglądarki internetowej.

HTTP (Hypertext Transfer Protocol)

Czym jest ?

- To protokół komunikacyjny (warstwa ISO Warstwa aplikacji) używany do przesyłania danych w sieci **WWW**.
- Pozwala on na wymianę informacji między serwerem a przeglądarką internetową (klientem).
- stanowi podstawę komunikacji danych w sieci World Wide Web , w której dokumenty hipertekstowe zawierają hiperłącza do innych zasobów.
- HTTP działa jako protokół żądanie-odpowiedź w modelu **klient-serwer**.

HTTPS (HyperText Transfer Protocol Secure)

Czym jest ?

- To bezpieczna wersja protokołu HTTP, który jest używany do przesyłania danych w internecie.
- Różnica między HTTP a **HTTPS** polega na tym, że HTTPS zapewnia szyfrowanie danych, co zwiększa bezpieczeństwo komunikacji.
- Jest szyfrowany przy użyciu Transport Layer Security (**TLS**) lub, dawniej, Secure Sockets Layer (**SSL**). Dlatego protokół jest również określany jako **HTTP over TLS/SSL**

URL

Czym jest ?

- **URL** (Uniform Resource Locator) to adres internetowy, który wskazuje lokalizację zasobu w Internecie.
- Składa się z kilku elementów, które umożliwiają określenie, gdzie znajduje się dana strona, plik lub inny zasób w sieci.

Przykładowy URL

<https://www.przyklad.com/katalog/book.html?lang=pl&font=roman#Chapter1>

URL

Podstawowe składowe URL

- <https://> - **Protokół** jaki zostanie wykorzystany do połączenia się z serwerem.
- www.przyklad.com - **Nazwa domeny (host)** identyfikuje serwer, na którym znajduje się zasób.
- [/katalog/book.html](http://www.przyklad.com/katalog/book.html) - **Ścieżka (path)** wskazuje dokładną lokalizację pliku lub zasobu na serwerze.
- [?lang=pl&font=roman](http://www.przyklad.com/?lang=pl&font=roman) - **Parametry zapytania (query)** służą do przekazywania dodatkowych danych do serwera.
- [#Chapter1](http://www.przyklad.com/#Chapter1) - **Fragment (fragment)** odnosi się do określonej sekcji w obrębie strony.

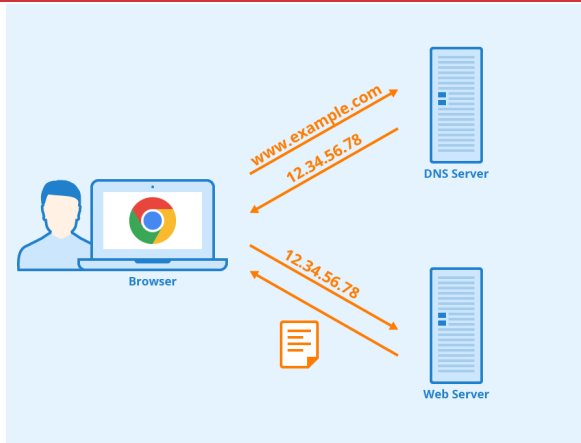
Domena

Czym jest ?

- **Domena** to unikalny adres, który pozwala użytkownikom znaleźć Twoją stronę internetową w Internecie.
- Jest to czytelna dla użytkownika nazwa, która zastępuje trudny do zapamiętania adres **IP** serwera, na którym hostowana jest dana strona.

Czym jest ?

- **DNS** (Domain Name System) to system, który umożliwia tłumaczenie nazw domenowych na **adresy IP**.
- Pełni funkcję "telefonicznej książki adresowej" dla internetu, umożliwiając użytkownikom korzystanie z łatwych do zapamiętania nazw zamiast trudnych adresów numerycznych.

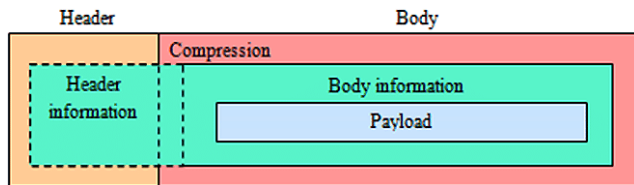


Rysunek 2: Działanie DNS

Request/Response (żądanie-odpowiedź)

Request/Response

- W protokole **HTTP** komunikacja między klientem a serwerem odbywa się poprzez wysyłanie żądań (**request**) i odbieranie odpowiedzi (**response**).



Rysunek 3: Struktura Request/Response

Nagłówki (Header)

Czym są ?

- **Nagłówki** w HTTP to metadane przesyłane wraz z żądaniem lub odpowiedzią między klientem a serwerem.
- Służą do przekazywania dodatkowych informacji o żądaniu lub odpowiedzi, takich jak typ danych, kodowanie, uwierzytelnianie czy informacje o cache.
- Są zazwyczaj niewidoczne dla użytkownika końcowego i są przetwarzane lub rejestrowane tylko przez serwer i aplikacje klienckie.

Metody HTTP

Metody HTTP

- Dla **request HTTP** definiuje metody, aby wskazać żądaną akcję, która ma zostać wykonana na zidentyfikowanym zasobie.
- Każdy klient może użyć dowolnej metody, a serwer można skonfigurować tak, aby obsługiwał dowolną kombinację metod.

Metody HTTP

Lista metod

- **GET** - Pobiera dane z serwera (np. stronę internetową).
- **POST** - Wysyła dane do serwera (np. formularz rejestracji).
- **PUT** - Aktualizuje zasób na serwerze lub tworzy nowy, jeśli nie istnieje.
- **PATCH** - Modyfikuje część zasobu na serwerze.
- **DELETE** - Usuwa zasób z serwera.
- **HEAD** - Podobne do GET, ale zwraca tylko nagłówki odpowiedzi, bez treści.
- **OPTIONS** - Zwraca dostępne metody dla danego zasobu.
- **TRACE** - Zwraca otrzymane żądanie w treści odpowiedzi.
- **CONNECT** - służy do nawiązywania tunelowanego połączenia z serwerem, zwykle w celu przesyłania zaszyfrowanych danych. (np. HTTPS przez proxy).

Statusy HTTP

Statusy HTTP

- W przypadku odpowiedzi serwera na żądanie klienta wraz z payload zostaje zwrócony **kodów statusu HTTP**.
- Informuje o wyniku żądania wysłanego do serwera.
- Dzięki nim klient wie, co się stało z jego żądaniem i jak powinien dalej postąpić.
- Statusy podzielone są na pięć kategorii:
 - **1xx** - Informacyjne (żądanie w toku)
 - **2xx** - Sukces (żądanie zakończone pomyślnie)
 - **3xx** - Przekierowania (klient powinien wykonać inne żądanie)
 - **4xx** - Błędy po stronie klienta (problem z żądaniem)
 - **5xx** - Błędy po stronie serwera (problem z przetworzeniem żądania)

Statusy HTTP

Często spotykane statusy HTTP

- **200 OK (Sukces)** - Oznacza, że żądanie zostało poprawnie obsłużone i serwer zwraca oczekiwane dane.
- **201 Created (Utworzono)** - Żądanie zakończone sukcesem, nowy zasób został utworzony.
- **301 Moved Permanently (Przekierowanie trwałe)** - Oznacza, że zasób został trwale przeniesiony na inny adres
- **400 Bad Request (Błędne żądanie)** - Oznacza, że serwer nie może przetworzyć żądania, ponieważ jest ono niepoprawnie sformułowane.
- **401 Unauthorized (Brak autoryzacji)** - Oznacza, że dostęp do zasobu wymaga uwierzytelnienia (np. zalogowania się).

Statusy HTTP

Często spotykane statusy HTTP

- **403 Forbidden (Brak uprawnień)** - Serwer odrzuca dostęp do zasobu, nawet jeśli użytkownik jest zalogowany.
- **404 Not Found (Nie znaleziono)** - Oznacza, że żądany zasób nie istnieje na serwerze.
- **500 Internal Server Error (Błąd serwera)** - Oznacza, że serwer napotkał błąd i nie może obsłużyć żądania.
- **429 Too Many Requests (Za dużo żądań)** - Klient wysłał zbyt wiele żądań w krótkim czasie i został tymczasowo zablokowany.

Co się dzieje, gdy wpisujesz adres URL w przeglądarkę i naciśniesz "Enter"

Wpisanie URL

- Kiedy wpisujesz **adres URL** w pasku adresu przeglądarki, przykładowo:
<https://www.przyklad.com/katalog/book.html?lang=pl&font=roman#Chapter1>
przeglądarka zaczyna przetwarzać ten wpis, a następnie przekształca go w odpowiednie zapytanie do serwera.

Rezolucja DNS

- Przeglądarka musi znaleźć serwer, na którym znajduje się strona.
- Żeby to zrobić, najpierw przeglądarka skontaktuje się z **serwerem DNS**, który rozpozna nazwę domeny i przekaże **adres IP**.
- Jeśli komputer nie ma w pamięci podręcznej tego adresu, zapytanie jest wysyłane do serwera DNS.

Co się dzieje, gdy wpisujesz adres URL w przeglądarkę i naciśniesz "Enter"

Nawiązywanie połączenia

- Proces: Używa protokołu **TCP** (dla HTTP/1 i /2), aby ustanowić połączenie z serwerem. Zaczyna się to tzw. "trójfazowym uściskiem ręki" (**Three-way handshake**):
 - **SYN**: Klient wysyła żądanie połączenia.
 - **SYN-ACK**: Serwer odpowiada potwierdzeniem.
 - **ACK**: Klient potwierdza, że połączenie zostało nawiązane.

Szyfrowanie

- Jeśli używasz protokołu **https**, to oznacza, że połączenie jest szyfrowane za pomocą **SSL/TLS**.
- Zanim przeglądarka wyśle jakiegolwiek musi wymienić **klucze szyfrowania** z serwerem.

Co się dzieje, gdy wpisujesz adres URL w przeglądarkę i naciśniesz "Enter"

Szyfrowanie cd.

- **Klucz publiczny** serwera służy do szyfrowania danych wysyłanych do serwera, a **prywatny klucz** służy do odszyfrowania odpowiedzi.
- Dzięki temu cały transfer danych między Twoją przeglądarką a serwerem jest zaszyfrowany, co uniemożliwia przechwycenie informacji przez osoby trzecie.

Żądanie HTTP

- Po nawiązaniu połączenia **TCP** (i opcjonalnie SSL/TLS dla HTTPS) przeglądarka wysyła do serwera żądanie HTTP (zazwyczaj **metodą GET**), aby zażądać określonego zasobu.

Co się dzieje, gdy wpisujesz adres URL w przeglądarkę i naciśniesz "Enter"

Odpowiedź serwera

- Serwer, otrzymawszy żądanie, przetwarza je i odpowiada na nie. Odpowiedź HTTP zawiera kod statusu (np. 200 OK) i dane, które klient (przeglądarka) chce otrzymać (np. kod HTML strony).

Renderowanie strony przez przeglądarkę

- Po otrzymaniu odpowiedzi z serwera, przeglądarka analizuje otrzymany **HTML**.
- Jeśli strona zawiera inne zasoby (takie jak obrazy, style CSS, skrypty JavaScript), przeglądarka wysyła kolejne zapytania HTTP, aby je pobrać.
- **DOM (Document Object Model):** HTML jest przetwarzany na strukturę drzewa DOM, która reprezentuje dokument jako obiekty, które można manipulować.

Co się dzieje, gdy wpisujesz adres URL w przeglądarkę i naciśniesz "Enter"

Renderowanie strony przez przeglądarkę

- **CSS:** Przeglądarka aplikuje style z CSS do elementów DOM, określając jak mają wyglądać na stronie.
- **JavaScript:** Skrypty JavaScript mogą być wykonywane, aby dodać interaktywność lub dynamicznie zmienić treść strony.

Strona jest wyświetlana

- Po przetworzeniu wszystkich zasobów (HTML, CSS, JavaScript, obrazy), przeglądarka rysuje stronę na ekranie, a Ty widzisz gotowy wynik na swojej witrynie.

- 1 Historia
- 2 Wprowadzenie
- 3 Front-end**
- 4 Back-end
- 5 Technologie wspierające

Front-end

Czym jest ?

- **Frontend** to część aplikacji internetowej lub strony WWW, którą widzi i z którą bezpośrednio interakcję ma użytkownik. Jest to warstwa prezentacji, odpowiedzialna za wygląd, układ oraz działanie interaktywnych elementów.

Główne technologie frontendu

- **HTML** (HyperText Markup Language) – struktura strony, definiowanie elementów, takich jak nagłówki, akapity, obrazy, formularze itp.
- **CSS** (Cascading Style Sheets) – stylizacja strony: kolory, czcionki, układ, animacje.
- **JavaScript** – interaktywność: obsługa zdarzeń, animacje, dynamiczne zmiany treści, walidacja formularzy.
- **jQuery** - biblioteka JavaScript, która ułatwia manipulowanie elementami HTML, obsługę zdarzeń, animacje oraz wykonywanie zapytań AJAX.

- 1 Historia
- 2 Wprowadzenie
- 3 Front-end
- 4 Back-end
- 5 Technologie wspierające

Back-end

Czym jest ?

- **Backend** to część aplikacji internetowej, która działa po stronie serwera i odpowiada za logikę biznesową, przechowywanie oraz przetwarzanie danych.
- Użytkownik nie ma bezpośredniego dostępu do backendu – komunikuje się z nim poprzez frontend, który wysyła żądania do serwera i odbiera odpowiedzi.

Języki programowania

- JavaScript (Express.js).
- Python (Django, Flask).
- PHP (Laravel).
- Java (Spring Boot).
- Ruby (Ruby on Rails).

Back-end

Bazy danych

- Relacyjne (MySQL, PostgreSQL).
- NoSQL (MongoDB, Firebase).

Serwery i hosting

- Apache, Nginx.
- Hosting w chmurze (AWS, Firebase, Vercel).

- 1 Historia
- 2 Wprowadzenie
- 3 Front-end
- 4 Back-end
- 5 Technologie wspierające**

Technologie wspierające

Architektury

- **REST API** (Representational State Transfer Application Programming Interface) to sposób komunikacji między systemami w architekturze REST. Jest używane do wymiany danych między aplikacjami, zazwyczaj poprzez protokół HTTP.
- **GraphQL** to nowoczesny język zapytań do API. Umożliwia klientom pobieranie tylko tych danych, które są im faktycznie potrzebne, eliminując nadmiarowe zapytania i optymalizując wydajność.

Technologie wspierające

CI/CD i DevOps

- **CI/CD** (Continuous Integration / Continuous Deployment lub Continuous Delivery) to praktyki automatyzacji w procesie tworzenia oprogramowania, które pomagają w szybkim i niezawodnym dostarczaniu aplikacji
- **Docker, Kubernetes** – konteneryzacja, wirtualizacja środowiska i oprogramowania.

Dziękuję za uwagę !